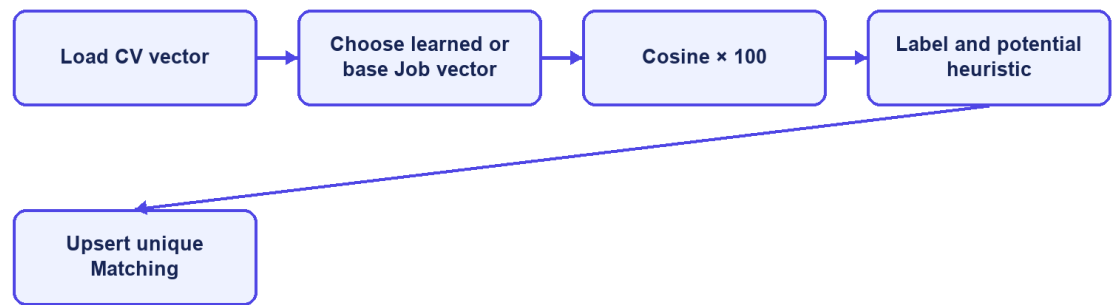


## Scoring and persistence



CareerFit IT AutoPilot — thesis diagram

Figure 4.5. Scoring and matching persistence flow

## 4.6 Feedback Learning with Rocchio

`FeedbackService.submitFeedback` resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching\_id, actor\_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT\_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD\_MATCH, POTENTIAL, and BAD\_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as  $q$ . Positive examples are CV vectors attached to GOOD\_MATCH or POTENTIAL feedback; negative examples come from BAD\_MATCH. The service computes term-wise centroids and applies:

$$q_{new} = 1.0q + 0.75 \text{ positive\_centroid} - 0.15 \text{ negative\_centroid}.$$

Negative weights are removed rather than persisted. The learned vector is stored in `learnedProfileVectorJson`. Every Matching associated with the Job is then marked `needsRecompute=true`. `AutomationScheduler.recomputeStaleMatchings` rescans these rows every 30 minutes and clears the marker only after successful scoring.